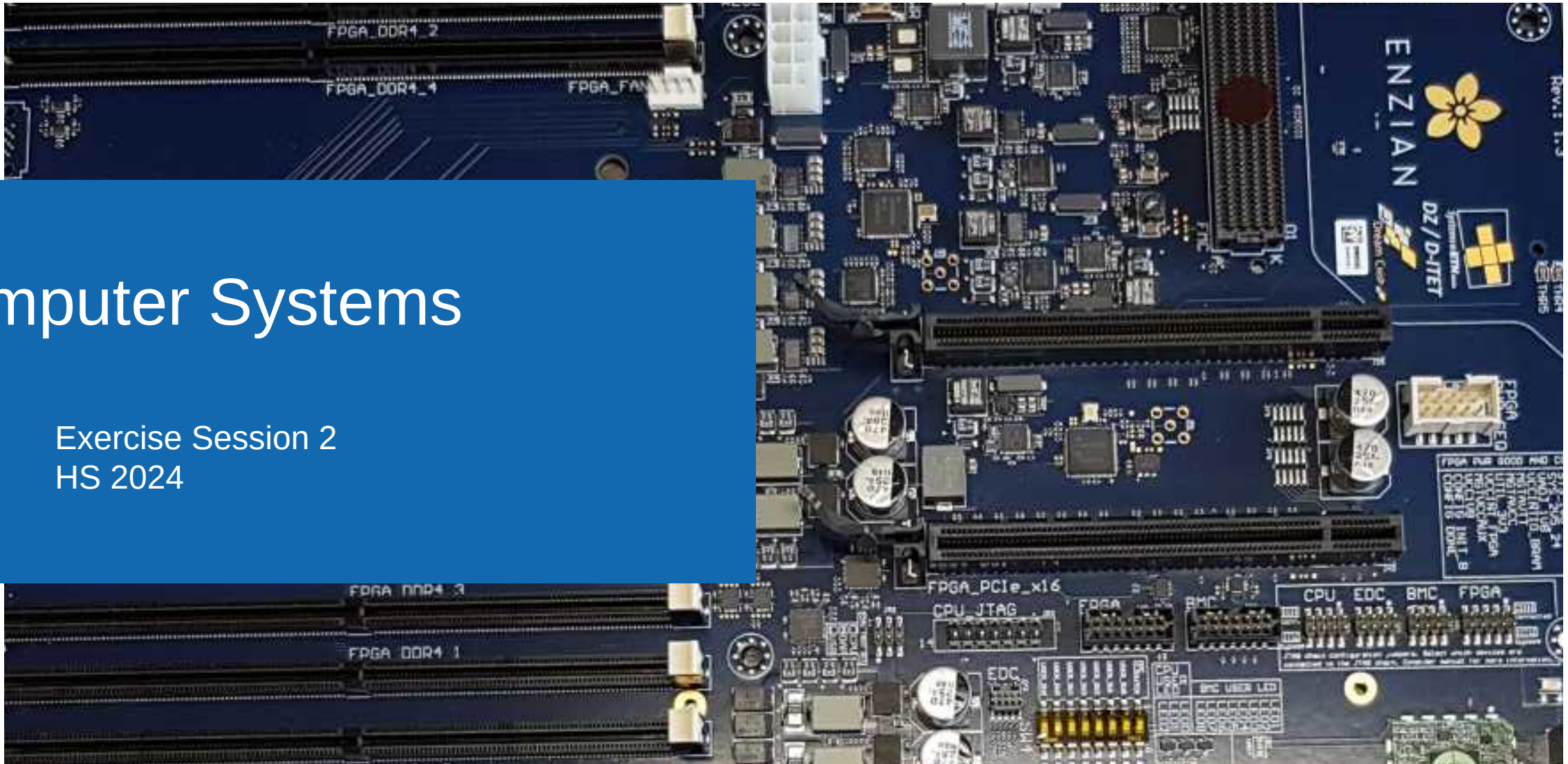


Computer Systems

Exercise Session 2
HS 2024



Program



1. Summary
2. Quiz
3. Assignment preview

Process

1. Instance of a program
2. Ingredients of a process:
 1. Virtual processor (address space, registers, program counter, instruction pointer)
 2. Program text (code)
 3. Program data (heap, stack)
 4. OS stuff (open files, sockets, CPU shares, security rights)
3. Executed in an “execution environment” made up of all the ingredients
4. Identified by PID (process ID)
5. Complete Software of a running Computer = running processes + kernel

spawn

1. Constructs a running process from scratch
2. In windows, CreateProcess() creates a new process to run a program
3. Spawn is complex
 1. Has to specify everything about the new environments including the command line.
 2. All rights a program needs must be granted in one go

fork/exec

1. Child is almost exact copy of calling parent
2. Return value from fork indicated which process we are in (0 : child, >0 : parent)
3. Can use full kernel API
 1. fork() : system call to create a copy of the current process
 2. Exec() : system call to change the program being run by the current process

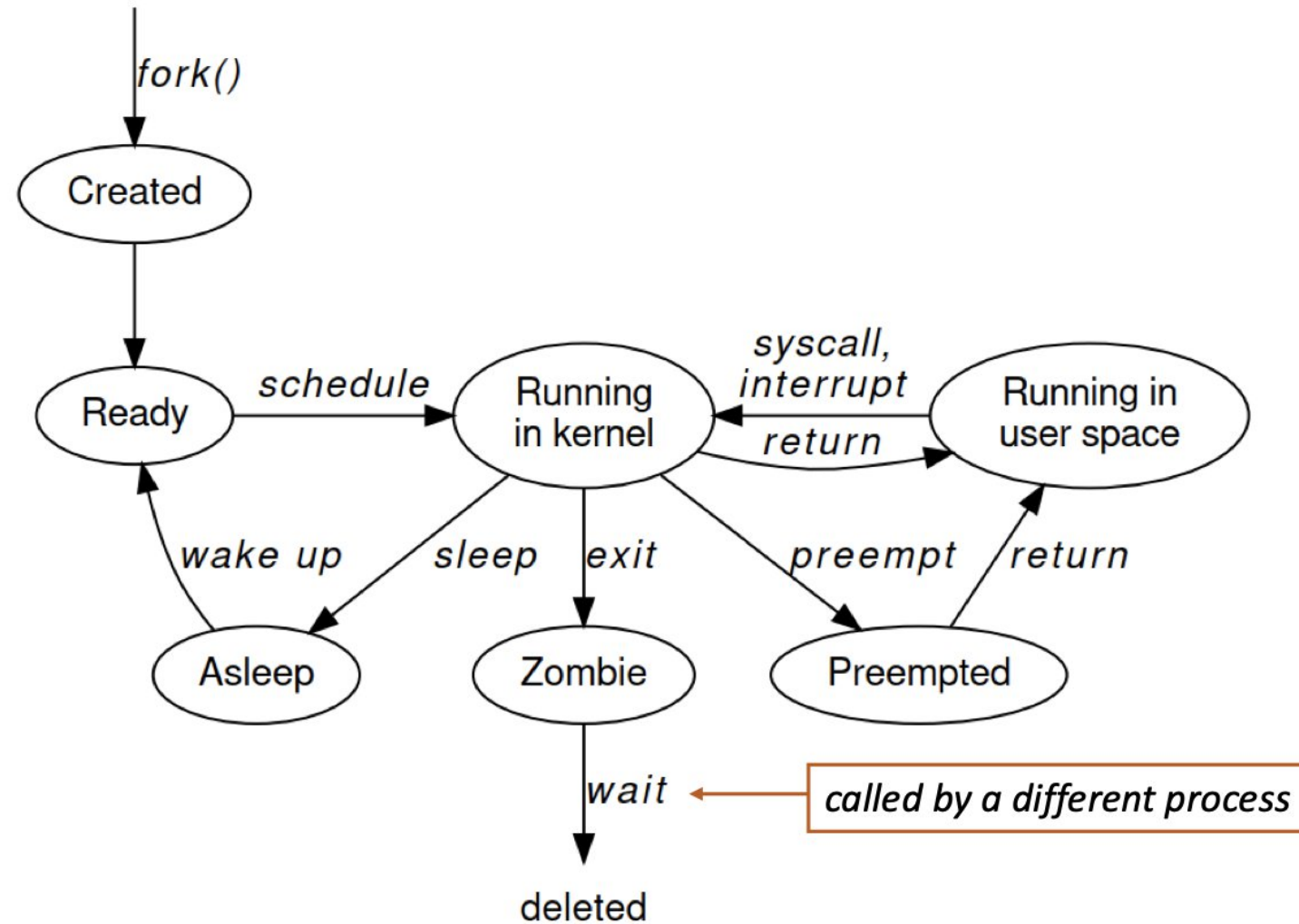
Example fork(), exec()

```
pid_t p = fork();  
if ( p < 0 ) {  
    // Error...  
    exit(-1);  
} else if ( p == 0 ) {  
    // We're in the child  
    execlp("/bin/ls", "ls", NULL);  
} else {  
    // We're a parent.  
    // p is the pid of the child  
    wait(NULL);  
    exit(0);  
}
```

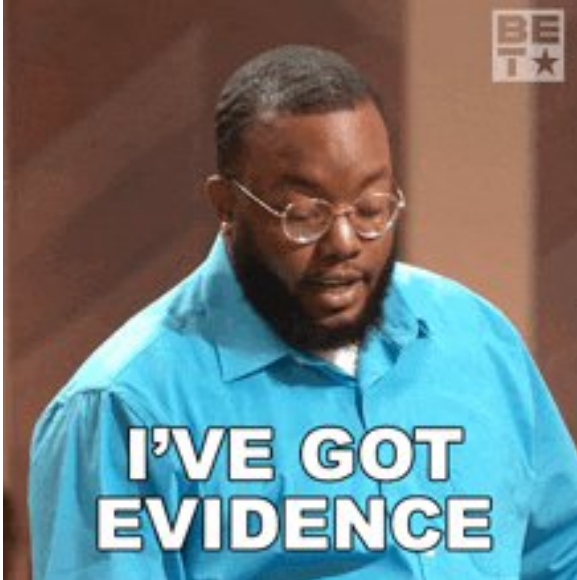
Return code from
fork() tells you
whether you're in the
parent or child
(cf. setjmp())

Child process can't
actually be cleaned
up until parent
"waits" for it.

Process Life Cycle



Linux Process Lifecycle



1. Zombies

```
( sleep 5 & wait $! ) & sleep 1; kill -STOP $!
```

2. Reparenting

```
( sleep 100 & echo $!; wait $! ) > /tmp/sleep.pid &  
echo ppid of 'sleep 100' is $!  
disown -h %1  
kill -KILL $!  
echo ppid of 'sleep 100' is $(ps -o ppid= -p $(cat /tmp/sleep.pid))
```

Watch what is happening in another terminal

```
watch --interval 1 "ps --forest --tty <tty> -o pid,ppid,cmd,stat"
```

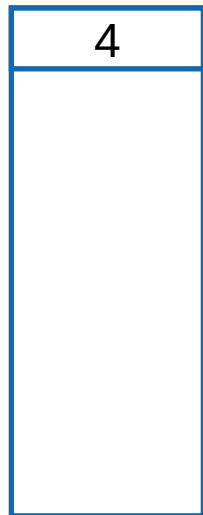
Watch what init process is doing

```
strace -p <PID>
```

The context switch

xv6: **Swch.S**

call Swch(&a->ctx, &b->ctx)



a

1: AA
2: BB
3: Swch(...)
4: CC

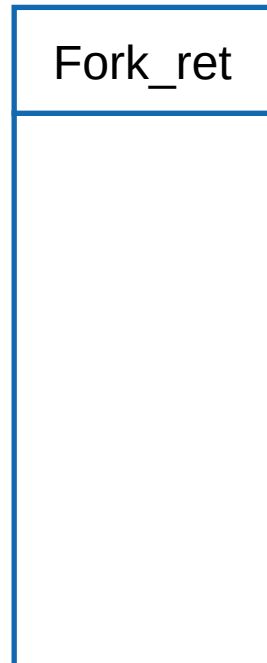


b

1: XX
2: YY
3: ZZ
4: Swch(...)
5: AA

xv6: **Swrch.S**

call Swrch(&scheduler->ctx, &new_proc->ctx)
where are we returning to ???



new_proc



Exam Time

What are the three hardware features without which a secure OS would not work ?
Why are they needed ?

What are the three hardware features without which a secure OS would not work ?
Why are they needed ?

- Privileged Instructions (kernel vs user space)
- Memory protection
- Timer interrupts

Explain how two processes can interact using `exit()` and `wait()` system calls

Recall that in traditional Unix-like systems, when the parent of a process dies, the process is reparented to “init”, the highest process in the hierarchy (process ID 1)

Why is this a better strategy than reparenting the process to its grandparent (i.e. to the parent of the process that has just died) ?

What is the output of the following C program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int val = 5;
    int pid;
    int wstatus;
    if (pid = fork()) {
        wait(&wstatus);
    } else {
        exit(val);
    }
    val ++;
    printf("%d\n", val);
    return val;
}
```

Quiz

What is the difference between a **process** and a **program**?



How are processes identified?



In what state is a process after it exited?



How does having only one kernel thread simplify the kernel?



Quiz

What is the difference between a **process** and a **program**?

- ▶ A process is a running instance of a program

How are processes identified?

- ▶

In what state is a process after it exited?

- ▶

How does having only one kernel thread simplify the kernel?

- ▶

Quiz

What is the difference between a **process** and a **program**?

- ▶ A process is a running instance of a program

How are processes identified?

- ▶ By the process ID (PID)

In what state is a process after it exited?

- ▶

How does having only one kernel thread simplify the kernel?

- ▶

Quiz

What is the difference between a **process** and a **program**?

- ▶ A process is a running instance of a program

How are processes identified?

- ▶ By the process ID (PID)

In what state is a process after it exited?

- ▶ Zombie state

How does having only one kernel thread simplify the kernel?

- ▶

Quiz

What is the difference between a **process** and a **program**?

- ▶ A process is a running instance of a program

How are processes identified?

- ▶ By the process ID (PID)

In what state is a process after it exited?

- ▶ Zombie state

Quiz

What is the difference between a **process** and a **program**?

- ▶ A process is a running instance of a program

How are processes identified?

- ▶ By the process ID (PID)

In what state is a process after it exited?

- ▶ Zombie state

Assignment 2

Note: Tasks 1.2 and 3 relate to Inter-Process Communication, which will only be discussed in the lecture this Friday (October 14). 🧑🏻 Start with the remaining exercises! ⏪

1. Task 1: get familiar with `fork()` and `exec()`
 - ▶ Hint : choose most useful `exec` function from linux man page
2. Task 2: Implementing cooperative user-level threads package
3. Task 3: Signals
 - ▶ Hint : `SIGHUP` signal: Hangup / death of controlling process
default action: terminate process